Master Computer Engineer
Software Trainer,
Senior Specialist, Advisor, Mentor,
Blockchain, Mobil, AI, Devops
Full Stack Developer &
Devops System Engineer

Hamit Mızrak



Spring Framework’e Giriş

Core Spring • MVC • Data • Security • Boot • Cloud • AI • RAG • Uçtan Uca Proje



1 IoC/DI

2 Web & Data

3 Security

4 Cloud

5 AI/RAG

Hazırlayan: Hamit MIZRAK • Kullanım: Eğitim/bootcamp/kurumsal ders planı

Kurs Haritası

100 başlık; temel kavramdan production-ready Spring uygulamasına.

1–30 • Temel Kavramlar: IoC, DI, Bean ve konfigürasyon zihniyetini kurmak.

31–45 • AOP, Event, Transaction ve Data: Kurumsal uygulamanın veri, transaction ve kesitsel davranış temelini oturtmak.

46–61 • Web, REST ve Hata Yönetimi: MVC, DispatcherServlet, REST, filter/interceptor ve exception akışını anlamak.

62–79 • Security, Test ve Entegrasyon: Kimlik doğrulama, yetkilendirme, test, cache, async ve reactive seçeneklerini öğretmek.

80–100 • Boot, Cloud, AI ve Final Proje: Spring Boot, mikroservisler, observability, Spring AI ve uçtan uca üretim senaryosu.

Eğitim yaklaşımı

Kavram → örnek → hata → production notu → mini görev akışıyla ilerler.

Ana çıktı

Öğrenci kurs sonunda Spring tabanlı REST + JPA + Security + Docker + AI/RAG projesi okuyup geliştirebilir.



Spring Framework nedir?

Temel bağlam: öğrencinin zihinsel modelini doğru kurmak.



Kurumsal Java uygulamaları için modüler, esnek ve geniş ekosistemli uygulama çatısıdır.

Ana gücü IoC container, dependency injection, AOP, transaction, web ve data entegrasyonudur.

Spring Boot ile birlikte modern Java backend geliştirmede en yaygın temel stacklerden biridir.

1 **Problem**

2 **Spring çözümü**

3 **Mini örnek**

4 **Hata analizi**

5 **Production notu**

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Neden Spring Framework öğreniyorum?

Temel bağlam: öğrencinin zihinsel modelini doğru kurmak.

Spring Boot kullanırken oluşan hataların çoğu Framework seviyesinde anlaşılır.
İş ilanlarında Spring Boot yazsa bile beklenen bilgi IoC, DI, transaction, JPA, MVC ve Security bilgisidir.
Kurumsal projelerde bakım, test, ölçekleme ve güvenlik kararları Spring temellerine dayanır.

1 **Problem**

2 **Spring çözümü**

3 **Mini örnek**

4 **Hata analizi**

5 **Production notu**

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Spring çalışma ve yaşam döngüsü büyük resmi

Temel bağlam: öğrencinin zihinsel modelini doğru kurmak.

Konfigürasyon okunur; bean definition oluşur; bean instance edilir; dependency injection yapılır.
Post processor, proxy, init callback ve kullanım aşamaları devreye girer.
Context kapanırken destroy callback ve resource temizliği yapılır.



1 Config

2 Bean Definition

3 Injection

4 Proxy/Init

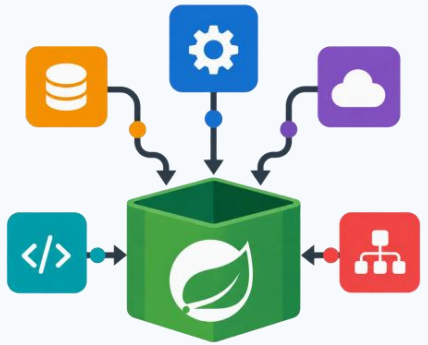
5 Destroy

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

DI / CDI farkı ve ilişki

Temel bağlam: öğrencinin zihinsel modelini doğru kurmak.

DI genel prensiptir: bağımlılığı sınıf üretmez, dışarıdan alır.
CDI, Jakarta EE dünyasında Contexts and Dependency Injection standardıdır.
Spring kendi IoC container modeline sahiptir; ayrıca standart @Inject gibi bazı annotation'ları destekleyebilir.



1 **Problem**

2 **Spring çözümü**

3 **Mini örnek**

4 **Hata analizi**

5 **Production notu**

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

XML nedir ve Spring’te neden bilmeliyim?

Temel bağlam: öğrencinin zihinsel modelini doğru kurmak.

XML, Extensible Markup Language; etiket tabanlı, hiyerarşik veri ve konfigürasyon formatıdır.
Eski Spring projelerinde bean tanımı, transaction, AOP ve datasource ayarları XML ile yapılmıştır.
Modern projede Java config/annotation yaygındır; ama legacy okuma için XML şarttır.



1 **Problem**

2 **Spring çözümü**

3 **Mini örnek**

4 **Hata analizi**

5 **Production notu**

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Eğitmen anlatım stratejisi

Temel bağlam: öğrencinin zihinsel modelini doğru kurmak.

Her başlıkta “problem ne?” sorusuyla başlanmalı.
Sonra Spring çözümü, küçük kod, hata senaryosu ve production notu verilmelidir.
Öğrencinin “bu annotation ne işe yarar?” yerine “runtime’da ne olur?” sorusunu sorması hedeflenmelidir.



1 **Problem**

2 **Spring çözümü**

3 **Mini örnek**

4 **Hata analizi**

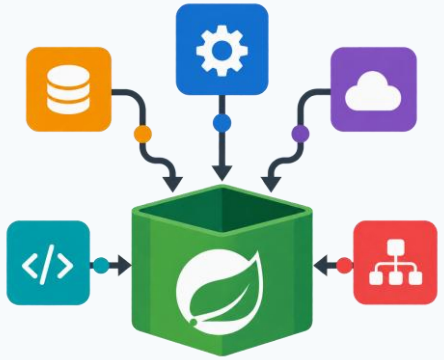
5 **Production notu**

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Modül 1

Temel Kavramlar

1–30 numaralı konu başlıkları



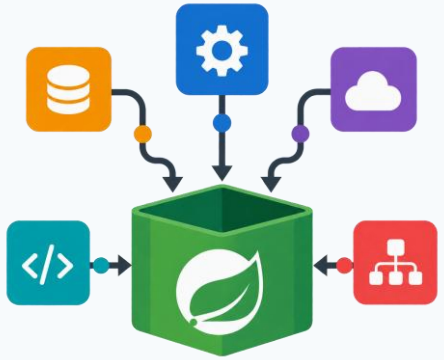
IoC, DI, Bean ve konfigürasyon zihniyetini kurmak.

1. Spring Ekosistemi ve Spring Framework Mimarisi
2. Spring Framework vs Spring Boot
3. IoC – Inversion of Control
4. DI – Dependency Injection
5. IoC Container
6. Bean Kavramı
7. ApplicationContext ve BeanFactory
8. Bean Lifecycle

Modül 1

Temel Kavramlar

1–30 numaralı konu başlıkları



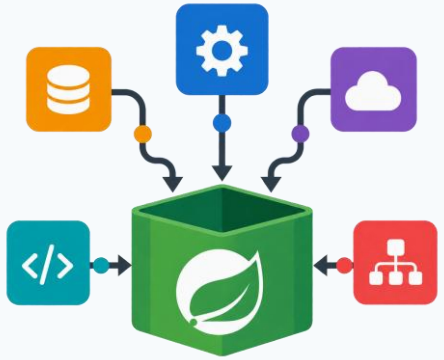
IoC, DI, Bean ve konfigürasyon zihniyetini kurmak.

1. Bean Scope
2. Component Scanning
3. Stereotype Annotations
4. @Component
5. @Service
6. @Repository
7. @Controller
8. Dependency Injection Yöntemleri

Modül 1

Temel Kavramlar

1–30 numaralı konu başlıkları



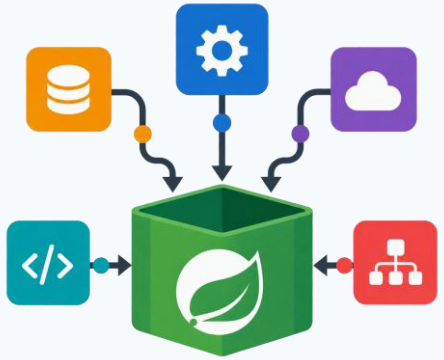
IoC, DI, Bean ve konfigürasyon zihniyetini kurmak.

1. Dependency Injection Yöntemleri
2. Constructor Injection
3. Setter Injection
4. Field Injection
5. @Autowired
6. @Qualifier
7. @Primary
8. Java-Based Configuration

Modül 1

Temel Kavramlar

1–30 numaralı konu başlıkları



IoC, DI, Bean ve konfigürasyon zihniyetini kurmak.

1. @Configuration
2. @Bean
3. XML-Based Configuration
4. Properties ve Environment Yönetimi
5. @Value
6. Spring Profiles – @Profile
7. Spring Expression Language – SpEL

1. Spring Ekosistemi ve Spring Framework Mimarisi

Temel Kavramlar

KAVRAM

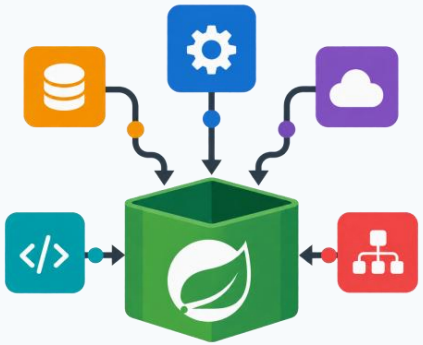
Spring; Core Container, AOP, Data Access, Web, Test ve entegrasyon modüllerinden oluşan kurumsal Java çatısıdır.

NEDEN ÖNEMLİ?

Amaç; nesne oluşturma, bağımlılık yönetimi, transaction, web ve güvenlik gibi tekrar eden işleri standartlaştırmaktır.

UYGULAMA / DİKKAT

Mimariyi öğrenmek, Spring Boot kullansanız bile alttaki kararları doğru vermenizi sağlar.



1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

2. Spring Framework vs Spring Boot

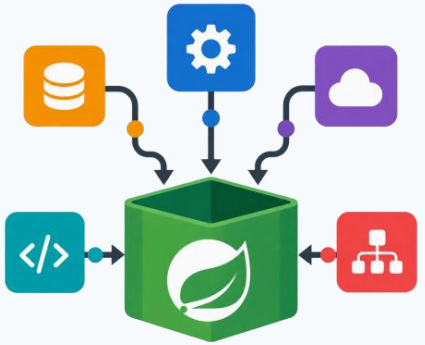
Temel Kavramlar

KAVRAM

Framework temel altyapıdır; Boot bu altyapıyı hızlı başlatan, otomatik yapılandıran üretkenlik katmanıdır.

NEDEN ÖNEMLİ?

Spring Boot, starter dependency ve auto configuration ile boilerplate azaltır.



UYGULAMA / DİKKAT

Framework bilinmeden Boot hataları “sihir” gibi görünür; Framework bilen geliştirici nedeni teşhis eder.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

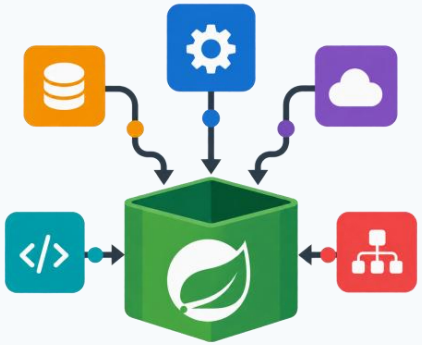
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

3. IoC – Inversion of Control

Temel Kavramlar

KAVRAM

Nesnenin yaşamını ve bağımlılıklarını uygulama kodu değil container yönetir.



NEDEN ÖNEMLİ?

new ile sıkı bağımlılık kurmak yerine nesneler container’dan alınır.

UYGULAMA / DİKKAT

IoC; test edilebilirlik, gevşek bağlılık ve merkezi konfigürasyon sağlar.

1 Tanım

2 Container

3 Bean/Proxy

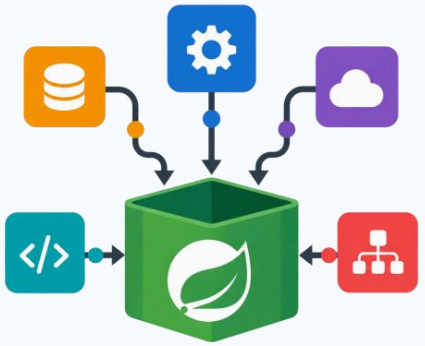
4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

4. DI – Dependency Injection

Temel Kavramlar



KAVRAM

Bir sınıfın ihtiyaç duyduğu nesneler dışarıdan verilir.

NEDEN ÖNEMLİ?

Constructor, setter veya field üzerinden uygulanabilir; kurumsal kodda constructor injection tercih edilir.

UYGULAMA / DİKKAT

DI ile sınıf kendi bağımlılığını üretmez, sadece iş kuralına odaklanır.

1 Tanım 2 Container 3 Bean/Proxy 4 Test 5 Prod

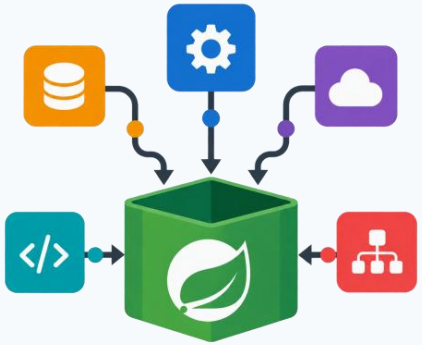
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

5. IoC Container

Temel Kavramlar

KAVRAM

Spring container bean tanımlarını okur, nesneleri üretir, bağlar, başlatır ve kapatır.



NEDEN ÖNEMLİ?

ApplicationContext modern ve zengin container arayüzüdür.

UYGULAMA / DİKKAT

Container; scope, lifecycle callback, event, environment ve resource erişimini yönetir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

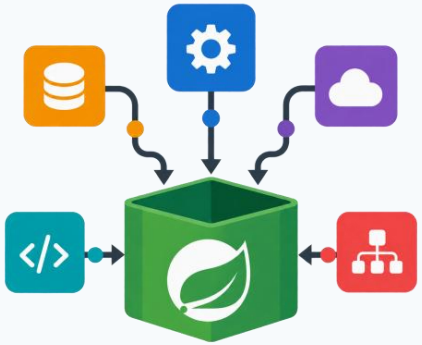
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

6. Bean Kavramı

Temel Kavramlar

KAVRAM

Bean, Spring container tarafından yönetilen nesnedir.



NEDEN ÖNEMLİ?

Bir sınıfın bean olması için component scan veya explicit @Bean tanımı gerekir.

UYGULAMA / DİKKAT

Bean adı, tipi, scope'u ve lifecycle davranışı container tarafından takip edilir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

7. ApplicationContext ve BeanFactory

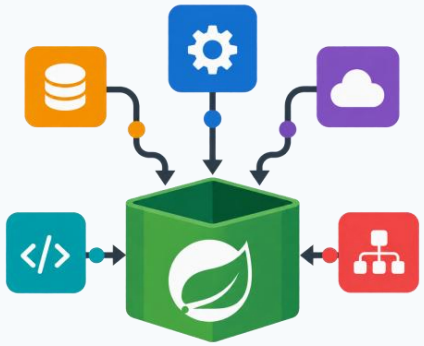
Temel Kavramlar

KAVRAM

BeanFactory temel IoC sözleşmesidir; ApplicationContext bunun üzerine event, i18n, resource ve AOP kabiliyetleri ekler.

NEDEN ÖNEMLİ?

Modern Spring uygulamalarında neredeyse daima ApplicationContext kullanılır.



UYGULAMA / DİKKAT

BeanFactory daha düşük seviye ve lazy odaklıdır; ApplicationContext kurumsal uygulama context'idir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

8. Bean Lifecycle

Temel Kavramlar

KAVRAM

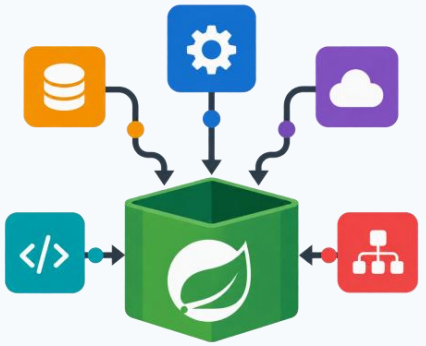
Tanım okuma, instance oluşturma, dependency injection, aware callback, post processor, init, kullanım ve destroy aşamalarından oluşur.

NEDEN ÖNEMLİ?

Lifecycle bilmek; bağlantı açma-kapatma, resource temizleme ve proxy davranışını anlamak için kritiktir.

UYGULAMA / DİKKAT

@PostConstruct ve @PreDestroy sık kullanılan lifecycle callback örnekleridir.



1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

9. Bean Scope

Temel Kavramlar

KAVRAM

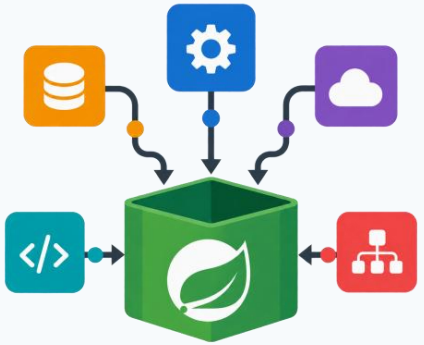
Scope, bean instance sayısını ve yaşam süresini belirler.

NEDEN ÖNEMLİ?

singleton varsayılandır; prototype her istek için yeni nesne üretir.

UYGULAMA / DİKKAT

web ortamında request, session, application gibi scope’lar vardır.



1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

10. Component Scanning

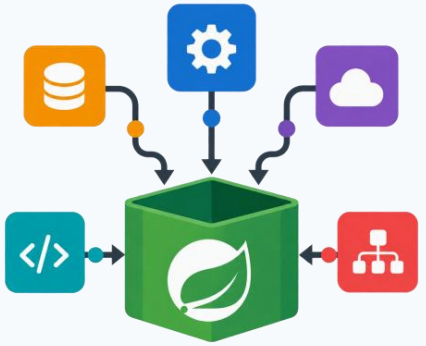
Temel Kavramlar

KAVRAM

Spring belirlenen package ağacını tarar ve stereotype annotation taşıyan sınıfları bean yapar.

NEDEN ÖNEMLİ?

Ana uygulama sınıfının package konumu tarama alanını belirler.



UYGULAMA / DİKKAT

Yanlış package yerleşimi bean bulunamadı hatalarının yaygın nedenidir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

11. Stereotype Annotations

Temel Kavramlar

KAVRAM

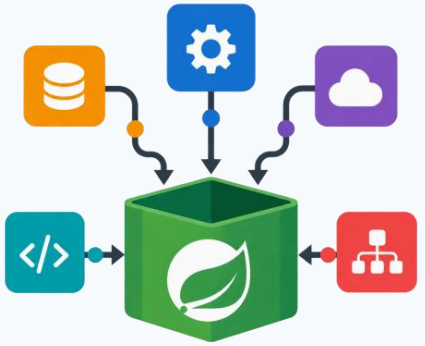
@Component genel bean; @Service iş katmanı; @Repository veri erişimi; @Controller web katmanı niyetini gösterir.

NEDEN ÖNEMLİ?

Teknik olarak çoğu component scan ile bean üretir; semantik fark mimari okunabilirlik sağlar.

UYGULAMA / DİKKAT

@Repository exception translation gibi ek davranışlar da sağlayabilir.



1 Tanım

2 Container

3 Bean/Proxy

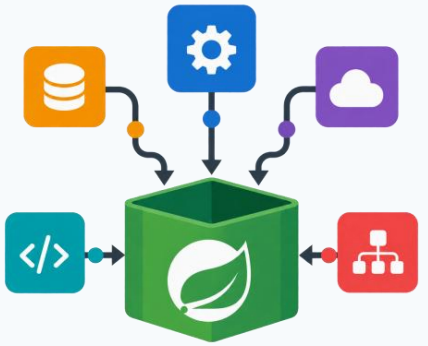
4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

12. @Component

Temel Kavramlar



KAVRAM

Genel amaçlı Spring bean annotation’ıdır.

NEDEN ÖNEMLİ?

Belirli katman anlamı yoksa yardımcı servis, converter, utility adapter gibi yapılarda kullanılır.

UYGULAMA / DİKKAT

Aşırı kullanılırsa mimari niyet kaybolur; uygun stereotype seçimi önemlidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

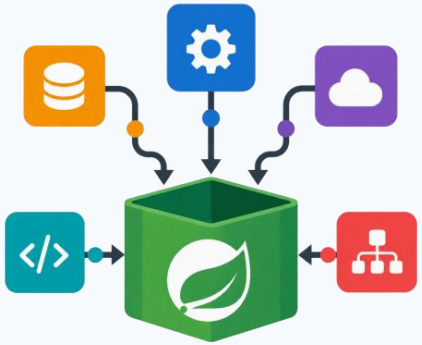
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

13. @Service

Temel Kavramlar

KAVRAM

Business logic katmanı için kullanılır.



NEDEN ÖNEMLİ?

Controller’dan gelen isteği iş kuralına çevirir, transaction sınırlarını taşıyabilir.

UYGULAMA / DİKKAT

Repository doğrudan controller’a bağlanmamalı; service katmanı arada olmalıdır.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

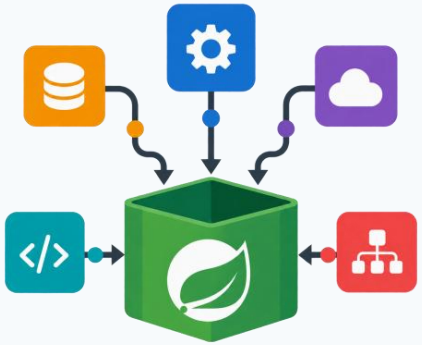
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

14. @Repository

Temel Kavramlar

KAVRAM

Veri erişim katmanını temsil eder.



NEDEN ÖNEMLİ?

DAO/repository sınıflarında kullanılır ve persistence exception translation desteği verir.

UYGULAMA / DİKKAT

SQL/JPA/NoSQL detaylarını service katmanından izole eder.

1 DTO → 2 Service → 3 Tx → 4 Repository → 5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

15. @Controller

Temel Kavramlar

KAVRAM

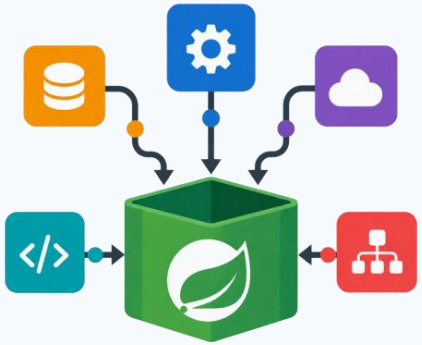
Spring MVC’de web controller sınıfıdır.

NEDEN ÖNEMLİ?

View döndüren klasik MVC uygulamalarında kullanılır.

UYGULAMA / DİKKAT

REST için çoğunlukla @RestController daha doğrudur.



1 HTTP

2 Controller

3 Service

4 Response

5 Error

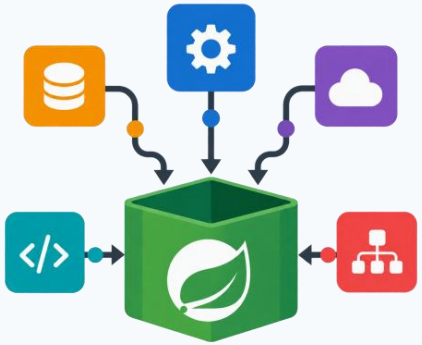
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

16. Dependency Injection Yöntemleri

Temel Kavramlar

KAVRAM

Constructor, setter ve field injection Spring’in bağımlılık verme yollarıdır.



NEDEN ÖNEMLİ?

Constructor injection zorunlu bağımlılıkları açıkça gösterir ve test kolaylığı sağlar.

UYGULAMA / DİKKAT

Setter opsiyonel bağımlılıklar için, field injection ise eğitim dışında genellikle kaçınılması gereken pratik olarak görülür.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

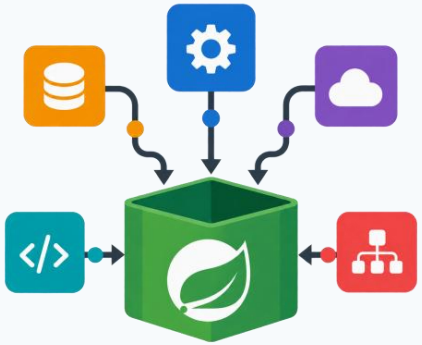
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

17. Constructor Injection

Temel Kavramlar

KAVRAM

Bağımlılıklar constructor parametresiyle verilir.



NEDEN ÖNEMLİ?

final field kullanımına izin verdiği için immutable ve test edilebilir sınıflar üretir.

UYGULAMA / DİKKAT

Döngüsel bağımlılıkları erken ortaya çıkarır.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

18. Setter Injection

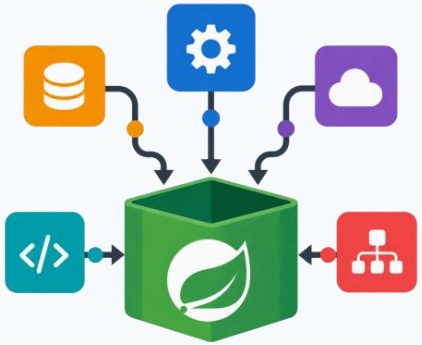
Temel Kavramlar

KAVRAM

Bağımlılık setter metodu üzerinden atanır.

NEDEN ÖNEMLİ?

Opsiyonel veya sonradan değişebilir bağımlılıklar için uygundur.



UYGULAMA / DİKKAT

Zorunlu bağımlılıkta nesne eksik state ile oluşabileceği için dikkat ister.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

19. Field Injection

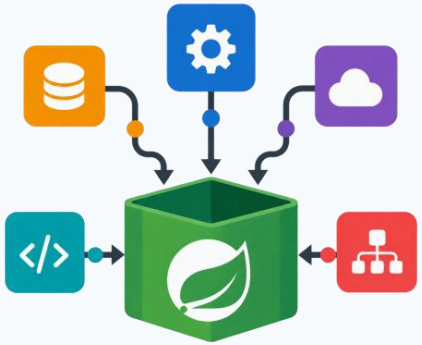
Temel Kavramlar

KAVRAM

Bağımlılık doğrudan field üzerine enjekte edilir.

NEDEN ÖNEMLİ?

Kod kısa görünür ama test, immutability ve açık bağımlılık açısından zayıftır.



UYGULAMA / DİKKAT

Üretim kodunda çoğunlukla constructor injection tercih edilmelidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

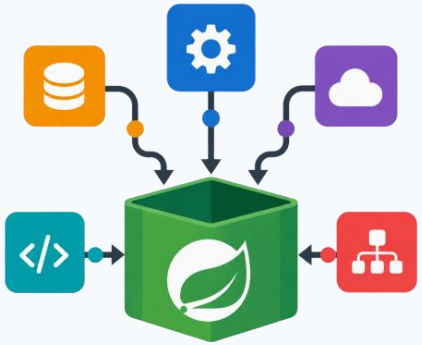
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

20. @Autowired

Temel Kavramlar

KAVRAM

Spring’in otomatik bağımlılık çözme annotation’ıdır.



NEDEN ÖNEMLİ?

Tip bazlı arama yapar; birden fazla aday varsa @Qualifier veya @Primary gerekir.

UYGULAMA / DİKKAT

Tek constructor varsa modern Spring’de @Autowired yazılmadan da injection yapılabilir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

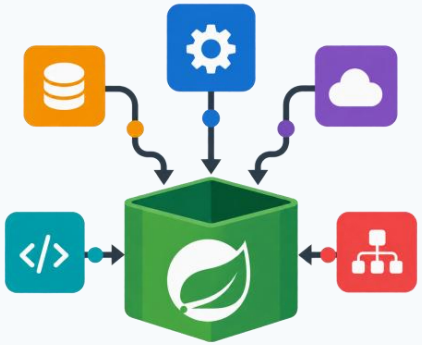
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

21. @Qualifier

Temel Kavramlar

KAVRAM

Aynı tipten birden fazla bean olduğunda hangi bean’in enjekte edileceğini belirtir.



NEDEN ÖNEMLİ?

Özellikle interface’in birden çok implementasyonu olduğunda kullanılır.

UYGULAMA / DİKKAT

Bean adlarını string ile bağladığı için refactor sırasında dikkat ister.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

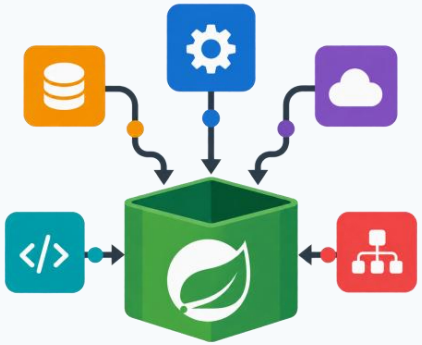
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

22. @Primary

Temel Kavramlar

KAVRAM

Birden fazla aday olduğunda varsayılan bean’i belirler.



NEDEN ÖNEMLİ?

@Qualifier yoksa primary bean seçilir.

UYGULAMA / DİKKAT

Genel varsayılan için iyidir; özel seçim gereken yerde @Qualifier daha net olur.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

23. Java-Based Configuration

Temel Kavramlar

KAVRAM

XML yerine Java sınıflarıyla bean ve ayar tanımlama yaklaşımıdır.



NEDEN ÖNEMLİ?

Tip güvenliği, refactor kolaylığı ve IDE desteği sağlar.

UYGULAMA / DİKKAT

Modern Spring ve Boot projelerinde ana konfigürasyon tarzıdır.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

24. @Configuration

Temel Kavramlar

KAVRAM

Bean tanımları içeren konfigürasyon sınıfını belirtir.



NEDEN ÖNEMLİ?

İçindeki @Bean metotları container tarafından yönetilir.

UYGULAMA / DİKKAT

Proxy davranışı sayesinde aynı @Bean metodu çağrılsa bile singleton sözleşmesi korunur.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

25. @Bean

Temel Kavramlar

KAVRAM

Bir nesneyi manuel olarak Spring bean yapmak için kullanılır.



NEDEN ÖNEMLİ?

Üçüncü parti sınıflar veya özel factory mantığı için idealdir.

UYGULAMA / DİKKAT

Metot adı varsayılan bean adı olur; name attribute ile özelleştirilebilir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

26. XML-Based Configuration

Temel Kavramlar

KAVRAM

Bean tanımlarının XML dosyasında yapıldığı eski ama halen anlaşılması değerli konfigürasyon biçimidir.

NEDEN ÖNEMLİ?

XML; tag tabanlı, hiyerarşik veri ve yapılandırma formatıdır.



UYGULAMA / DİKKAT

Legacy projelerde, entegrasyon sistemlerinde ve bazı kurumsal yapılarda karşınıza çıkabilir.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

27. Properties ve Environment Yönetimi

Temel Kavramlar

KAVRAM

Ayarları koddan ayırıp properties/yaml/env değişkenleriyle yönetmektir.



NEDEN ÖNEMLİ?

DB URL, port, token, profil ve feature flag gibi değerler environment üzerinden okunur.

UYGULAMA / DİKKAT

12-factor prensibine uygun production deployment için zorunludur.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

28. @Value

Temel Kavramlar

KAVRAM

Properties veya SpEL ifadelerini field/constructor parametresine bağlar.



NEDEN ÖNEMLİ?

Basit tekil değerler için uygundur.

UYGULAMA / DİKKAT

Çok sayıda ayar için @ConfigurationProperties daha düzenli tercih edilir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

29. Spring Profiles – @Profile

Temel Kavramlar

KAVRAM

Farklı ortamlar için farklı bean veya konfigürasyon seçmeyi sağlar.



NEDEN ÖNEMLİ?

dev, test, prod gibi ortam ayrımı yapılır.

UYGULAMA / DİKKAT

Yanlış profil production'da kritik bağlantı veya güvenlik hatalarına yol açabilir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

30. Spring Expression Language – SpEL

Temel Kavramlar

KAVRAM

Runtime expression değerlendirme dilidir.



NEDEN ÖNEMLİ?

Bean property, method call, collection, condition ve security ifadelerinde kullanılabilir.

UYGULAMA / DİKKAT

Güçlüdür ancak karmaşık iş kurallarını annotation içine gömmek okunabilirliği azaltır.

1 Tanım 2 Container 3 Bean/Proxy 4 Test 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Modül 2

AOP, Event, Transaction ve Data

31–45 numaralı konu başlıkları



Kurumsal uygulamanın veri, transaction ve kesitsel davranış temelini oturtmak.

- 31. Spring AOP – Aspect Oriented Programming
- 32. Aspect, Advice, Pointcut ve Join Point
- 33. @Aspect
- 34. Spring Events
- 35. Transaction Management
- 36. @Transactional
- 37. Spring JDBC
- 38. JdbcTemplate
- 39. Spring ORM
- 40. JPA ve Hibernate Entegrasyonu
- 41. Spring Data JPA
- 42. Repository Mimarisi
- 43. CRUD İşlemleri
- 44. Entity Relationships • 45. Validation

31. Spring AOP – Aspect Oriented Programming

AOP, Event, Transaction ve Data

KAVRAM

Logging, transaction, security gibi kesitsel konuları iş kodundan ayırır.



NEDEN ÖNEMLİ?

Proxy tabanlı yaklaşım ile metot çağrılarına ek davranış eklenir.

UYGULAMA / DİKKAT

AOP, core business logic'in sade kalmasını sağlar.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

32. Aspect, Advice, Pointcut ve Join Point

AOP, Event, Transaction ve Data

KAVRAM

Aspect kesitsel modüldür; advice çalışacak ek davranıştır; pointcut nerede çalışacağını seçer; join point yakalanan noktadır.

NEDEN ÖNEMLİ?

Spring AOP çoğunlukla method execution join point kullanır.



UYGULAMA / DİKKAT

Yanlış pointcut beklenmeyen metotlarda performans ve davranış sorunları doğurabilir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

33. @Aspect

AOP, Event, Transaction ve Data

KAVRAM

Bir sınıfı AOP aspect olarak işaretler.



NEDEN ÖNEMLİ?

@Before, @After, @Around gibi advice metotları içerir.

UYGULAMA / DİKKAT

@Around en güçlü ama en riskli advice türüdür; proceed çağrısı unutulmamalıdır.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

34. Spring Events

AOP, Event, Transaction ve Data

KAVRAM

Publisher-listener modeliyle gevşek bağlı olay iletişimi sağlar.



NEDEN ÖNEMLİ?

ApplicationEventPublisher ile event yayınlanır, @EventListener ile dinlenir.

UYGULAMA / DİKKAT

Email gönderme, audit, bildirim gibi yan etkiler için kullanışlıdır.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

35. Transaction Management

AOP, Event, Transaction ve Data

KAVRAM

Birden çok DB işleminin atomik bütün olarak yönetilmesidir.



NEDEN ÖNEMLİ?

Commit başarılı sonucu, rollback hata durumunu temsil eder.

UYGULAMA / DİKKAT

Kurumsal veri tutarlılığı için transaction sınırları doğru katmanda kurulmalıdır.

1 DTO **2** Service **3** Tx **4** Repository **5** DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

36. @Transactional

AOP, Event, Transaction ve Data

KAVRAM

Metot veya sınıf üzerinde transaction davranışını tanımlar.



NEDEN ÖNEMLİ?

Runtime exception durumunda varsayılan rollback davranışı vardır.

UYGULAMA / DİKKAT

Self-invocation, private method ve proxy sınırları sık yapılan hatalardır.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

37. Spring JDBC

AOP, Event, Transaction ve Data

KAVRAM

JDBC kullanımını Spring altyapısıyla sadeleştirir.



NEDEN ÖNEMLİ?

Connection açma-kapama ve exception dönüşümü gibi tekrarları azaltır.

UYGULAMA / DİKKAT

ORM istemeyen ama SQL kontrolü isteyen projeler için uygundur.

1 DTO → 2 Service → 3 Tx → 4 Repository → 5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

38. JdbcTemplate

AOP, Event, Transaction ve Data

KAVRAM

SQL çalıştırma, parametre bağlama ve result mapping işlemlerini kolaylaştırır.



NEDEN ÖNEMLİ?

Boilerplate JDBC kodunu ciddi azaltır.

UYGULAMA / DİKKAT

SQL performans kontrolü gereken servislerde halen değerli bir seçenektir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

39. Spring ORM

AOP, Event, Transaction ve Data

KAVRAM

Hibernate/JPA gibi ORM araçlarını Spring transaction ve bean modeliyle entegre eder.



NEDEN ÖNEMLİ?

Session/EntityManager yönetimini kolaylaştırır.

UYGULAMA / DİKKAT

ORM soyutlama sağlasa da N+1, lazy loading ve transaction sınırları bilinmelidir.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

40. JPA ve Hibernate Entegrasyonu

AOP, Event, Transaction ve Data

KAVRAM

JPA standart arayüz; Hibernate en yaygın implementasyondur.



NEDEN ÖNEMLİ?

EntityManager, persistence context ve entity lifecycle ana kavramlardır.

UYGULAMA / DİKKAT

Spring, repository ve transaction ile JPA kullanımını üretim seviyesinde kolaylaştırır.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

41. Spring Data JPA

AOP, Event, Transaction ve Data

KAVRAM

Repository arayüzlerinden otomatik veri erişim implementasyonu üretir.



NEDEN ÖNEMLİ?

Method name query, @Query, pagination ve specification destekler.

UYGULAMA / DİKKAT

Hızlı CRUD sağlar ama karmaşık sorgularda SQL/JPA bilgisi hâlâ gerekir.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

42. Repository Mimarisi

AOP, Event, Transaction ve Data

KAVRAM

Veri erişimini interface tabanlı soyutlar.



NEDEN ÖNEMLİ?

Service katmanı persistence detaylarını bilmeden çalışır.

UYGULAMA / DİKKAT

Repository sadece veri erişiminden sorumlu olmalı, iş kuralı taşımamalıdır.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

43. CRUD İşlemleri

AOP, Event, Transaction ve Data

KAVRAM

Create, Read, Update, Delete temel veri operasyonlarıdır.



NEDEN ÖNEMLİ?

REST endpoint, service transaction ve repository çağrısı zinciriyle uygulanır.

UYGULAMA / DİKKAT

Soft delete, audit ve validation kurumsal CRUD için ek gereksinimlerdir.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

44. Entity Relationships

AOP, Event, Transaction ve Data

KAVRAM

Entity’ler arası OneToOne, OneToMany, ManyToOne, ManyToMany ilişkilerini ifade eder.



NEDEN ÖNEMLİ?

Owning side, cascade, fetch type ve orphan removal kritik ayarlardır.

UYGULAMA / DİKKAT

Yanlış ilişki modelleme performans ve veri bütünlüğü sorunları üretir.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

45. Validation

AOP, Event, Transaction ve Data

KAVRAM

Gelen veriyi iş kuralı öncesi doğrulama sürecidir.



NEDEN ÖNEMLİ?

Jakarta Bean Validation ile @NotNull, @Email, @Size gibi annotation’lar kullanılır.

UYGULAMA / DİKKAT

DTO seviyesinde validation API sözleşmesini korur.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Modül 3

Web, REST ve Hata Yönetimi

46–61 numaralı konu başlıkları



MVC, DispatcherServlet, REST, filter/interceptor ve exception akışını anlamak.

- 46. Spring MVC
- 47. MVC Mimarisi
- 48. DispatcherServlet
- 49. Controller Yapısı
- 50. Request Mapping
- 51. Request Parameter / Path Variable
- 52. Form ve Model Yönetimi

Modül 3

Web, REST ve Hata Yönetimi

46–61 numaralı konu başlıkları



MVC, DispatcherServlet, REST, filter/interceptor ve exception akışını anlamak.

- 53. REST API Geliştirme
- 54. @RestController
- 55. @RequestBody
- 56. ResponseEntity
- 57. Exception Handling
- 58. @ControllerAdvice
- 59. Global Exception Handling
- 60. Interceptor
- 61. Filter

46. Spring MVC

Web, REST ve Hata Yönetimi

KAVRAM

Servlet tabanlı web framework’tür.



NEDEN ÖNEMLİ?

Controller, model, view veya REST response akışını yönetir.

UYGULAMA / DİKKAT

DispatcherServlet merkezli request handling modeli üzerine kuruludur.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

47. MVC Mimarisi

Web, REST ve Hata Yönetimi

KAVRAM

Model veriyi, View sunumu, Controller istek akışını temsil eder.



NEDEN ÖNEMLİ?

Sorumluluk ayrımı sayesinde frontend/backend veya server-side view yapıları düzenlenir.

UYGULAMA / DİKKAT

REST projelerinde View yerini JSON response alır.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

48. DispatcherServlet

Web, REST ve Hata Yönetimi



KAVRAM

Spring MVC’nin front controller bileşenidir.

NEDEN ÖNEMLİ?

Tüm requestleri karşılar, handler mapping, adapter, binding ve view/response sürecini yönetir.

UYGULAMA / DİKKAT

MVC akışını anlamanın en merkezi kavramıdır.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

49. Controller Yapısı

Web, REST ve Hata Yönetimi

KAVRAM

HTTP isteğini alır, input doğrular, service çağırır ve response döner.



NEDEN ÖNEMLİ?

Controller’da iş kuralı değil orkestrasyon bulunmalıdır.

UYGULAMA / DİKKAT

İnce controller, test edilebilir ve sürdürülebilir mimari sağlar.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

50. Request Mapping

Web, REST ve Hata Yönetimi

KAVRAM

URL, HTTP method, consumes/produces gibi kriterlere göre handler eşler.



NEDEN ÖNEMLİ?

@GetMapping, @PostMapping gibi kısayollar okunabilirlik sağlar.

UYGULAMA / DİKKAT

Endpoint tasarımında isimlendirme ve HTTP semantiği önemlidir.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

51. Request Parameter / Path Variable

Web, REST ve Hata Yönetimi

KAVRAM

RequestParam query string değerlerini, PathVariable URL path içindeki değerleri alır.



NEDEN ÖNEMLİ?

Filtreleme/sıralama için query param; kaynak kimliği için path variable kullanılır.

UYGULAMA / DİKKAT

Doğru ayrım REST tasarımını okunabilir kılar.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

52. Form ve Model Yönetimi

Web, REST ve Hata Yönetimi

KAVRAM

Server-side MVC’de form verisi binding, validation ve model ile view’a taşınır.



NEDEN ÖNEMLİ?

ModelAttribute form nesnesini bağlar.

UYGULAMA / DİKKAT

Hata mesajları ve validation sonuçları kullanıcı deneyimi için önemlidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

53. REST API Geliştirme

Web, REST ve Hata Yönetimi

KAVRAM

Kaynak odaklı, HTTP tabanlı API geliştirme yaklaşımıdır.

NEDEN ÖNEMLİ?

JSON, status code, idempotency ve endpoint standardı temel konulardır.



UYGULAMA / DİKKAT

REST API dış sistemler, frontend ve mobil istemciler için sözleşme üretir.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

54. @RestController

Web, REST ve Hata Yönetimi

KAVRAM

@Controller + @ResponseBody birleşimidir.

NEDEN ÖNEMLİ?

Metot dönüşleri view değil doğrudan HTTP response body olur.



UYGULAMA / DİKKAT

JSON API geliştirmede standart controller annotation'ıdır.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

55. @RequestBody

Web, REST ve Hata Yönetimi

KAVRAM

HTTP request body içindeki JSON/XML veriyi Java nesnesine dönüştürür.



NEDEN ÖNEMLİ?

DTO ile birlikte kullanılmalıdır.

UYGULAMA / DİKKAT

Content-Type ve deserialization hataları sık görülür.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

56. ResponseEntity

Web, REST ve Hata Yönetimi

KAVRAM

Body, status code ve header değerlerini birlikte kontrol etmeyi sağlar.



NEDEN ÖNEMLİ?

201 Created, 204 No Content, 400 Bad Request gibi yanıtlar net üretilir.

UYGULAMA / DİKKAT

API sözleşmesini profesyonel hale getirir.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

57. Exception Handling

Web, REST ve Hata Yönetimi

KAVRAM

Hataların kontrollü yakalanıp anlamlı response'a çevrilmesidir.



NEDEN ÖNEMLİ?

Stack trace kullanıcıya gösterilmez; standart hata modeli döndürülür.

UYGULAMA / DİKKAT

Tekrarlı try-catch yerine merkezi hata yönetimi tercih edilir.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

58. @ControllerAdvice

Web, REST ve Hata Yönetimi

KAVRAM

Controller’lar için merkezi exception, binding ve model advice tanımlar.



NEDEN ÖNEMLİ?

@ExceptionHandler ile global hata cevapları üretir.

UYGULAMA / DİKKAT

REST API hata standardizasyonunun ana mekanizmasıdır.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

59. Global Exception Handling

Web, REST ve Hata Yönetimi

KAVRAM

Tüm uygulama için tek tip hata yanıtı üretir.



NEDEN ÖNEMLİ?

Hata kodu, mesaj, timestamp, path ve validation detayları standardize edilir.

UYGULAMA / DİKKAT

Frontend ve mobil ekipler için öngörülebilir API davranışı sağlar.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

60. Interceptor

Web, REST ve Hata Yönetimi

KAVRAM

Spring MVC seviyesinde request öncesi/sonrası davranış ekler.



NEDEN ÖNEMLİ?

HandlerInterceptor ile auth kontrolü, logging, locale gibi işler yapılabilir.

UYGULAMA / DİKKAT

Controller seçildikten sonra devreye girdiği için Filter’dan farklıdır.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

61. Filter

Web, REST ve Hata Yönetimi

KAVRAM

Servlet seviyesinde request/response zincirinde çalışır.



NEDEN ÖNEMLİ?

Security, CORS, encoding ve low-level request işleme için uygundur.

UYGULAMA / DİKKAT

Controller mapping’den önce çalışır; kapsamı interceptor’dan daha geneldir.

1 HTTP

2 Controller

3 Service

4 Response

5 Error

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Modül 4

Security, Test ve Entegrasyon

62–79 numaralı konu başlıkları



Kimlik doğrulama, yetkilendirme, test, cache, async ve reactive seçeneklerini öğretmek.

- 62. Spring Security
- 63. Authentication
- 64. Authorization
- 65. Role ve Authority
- 66. Password Encoding
- 67. Session-Based Authentication
- 68. JWT Authentication

Modül 4

Security, Test ve Entegrasyon

62–79 numaralı konu başlıkları



Kimlik doğrulama, yetkilendirme, test, cache, async ve reactive seçeneklerini öğretmek.

- 69. CORS / CSRF
- 70. Spring Testing
- 71. JUnit Entegrasyonu
- 72. Mockito
- 73. Integration Testing
- 74. Spring Cache
- 75. Scheduling
- 76. Async Programming • 77. Spring Mail • 78. Spring WebSocket • 79. Spring WebFlux ve Reactive Programming

62. Spring Security

Security, Test ve Entegrasyon

KAVRAM

Authentication, authorization, CSRF, session, filter chain gibi güvenlik bileşenlerini yönetir.

NEDEN ÖNEMLİ?

Varsayılan güvenlik sağlayarak açıkları azaltır.



UYGULAMA / DİKKAT

Doğru yapılandırılmazsa erişim, CORS veya CSRF sorunları doğurabilir.

1 Request

2 Filter

3 Auth

4 Yetki

5 Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

63. Authentication

Security, Test ve Entegrasyon

KAVRAM

Kullanıcının kim olduğunu doğrulama sürecidir.

NEDEN ÖNEMLİ?

Username/password, JWT, OAuth2 gibi yöntemler kullanılabilir.



UYGULAMA / DİKKAT

Sonuçta SecurityContext içinde Authentication nesnesi oluşur.

1 Request

2 Filter

3 Auth

4 Yetki

5 Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

64. Authorization

Security, Test ve Entegrasyon

KAVRAM

Doğrulanmış kullanıcının hangi kaynağa erişebileceğini belirler.



NEDEN ÖNEMLİ?

Role, authority, method security ve URL security ile uygulanır.

UYGULAMA / DİKKAT

Authentication kimliktir; authorization yetkidir.

1 Request

2 Filter

3 Auth

4 Yetki

5 Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

65. Role ve Authority

Security, Test ve Entegrasyon

KAVRAM

Authority spesifik izin; role çoğunlukla ROLE_ prefix'li yetki grubudur.



NEDEN ÖNEMLİ?

hasRole ve hasAuthority farkı bilinmelidir.

UYGULAMA / DİKKAT

İyi tasarımda role iş unvanı değil erişim politikasıdır.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

66. Password Encoding

Security, Test ve Entegrasyon

KAVRAM

Şifrelerin düz metin değil hashlenmiş saklanmasıdır.



NEDEN ÖNEMLİ?

BCryptPasswordEncoder yaygın tercihtir.

UYGULAMA / DİKKAT

Encode tek yönlüdür; login sırasında ham şifre hash ile karşılaştırılır.

1 Request

2 Filter

3 Auth

4 Yetki

5 Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

67. Session-Based Authentication

Security, Test ve Entegrasyon

KAVRAM

Login sonrası server tarafında session tutulur.



NEDEN ÖNEMLİ?

Klasik web uygulamalarında pratik ve güvenlidir.

UYGULAMA / DİKKAT

Yatay ölçeklemede sticky session veya merkezi session store ihtiyacı doğabilir.

1 Request

2 Filter

3 Auth

4 Yetki

5 Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

68. JWT Authentication

Security, Test ve Entegrasyon

KAVRAM

Token tabanlı stateless kimlik doğrulamadır.



NEDEN ÖNEMLİ?

Access token request header ile taşınır.

UYGULAMA / DİKKAT

Token süresi, refresh token, imza ve blacklist stratejisi dikkat ister.

1 Request **2** Filter **3** Auth **4** Yetki **5** Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

69. CORS / CSRF

Security, Test ve Entegrasyon

KAVRAM

CORS tarayıcı kaynak paylaşım politikasını; CSRF kullanıcı oturumunu kötüye kullanma saldırısını ilgilendirir.

NEDEN ÖNEMLİ?

REST frontend ayrı domainde ise CORS ayarı gerekir.



UYGULAMA / DİKKAT

Session tabanlı sistemlerde CSRF koruması önemlidir; stateless JWT'de yaklaşım farklıdır.

1 Request

2 Filter

3 Auth

4 Yetki

5 Response

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

70. Spring Testing

Security, Test ve Entegrasyon

KAVRAM

Unit, slice ve integration test seviyelerini destekler.



NEDEN ÖNEMLİ?

Spring TestContext Framework context yönetimi sağlar.

UYGULAMA / DİKKAT

Amaç hızlı, güvenilir ve izole doğrulama yapmaktır.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

71. JUnit Entegrasyonu

Security, Test ve Entegrasyon

KAVRAM

JUnit test runner ve assertion altyapısıdır.

NEDEN ÖNEMLİ?

@SpringBootTest, @WebMvcTest gibi annotation’larla Spring testleri çalıştırılır.



UYGULAMA / DİKKAT

Test isimlendirme ve fixture düzeni bakım maliyetini etkiler.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

72. Mockito

Security, Test ve Entegrasyon

KAVRAM

Mock nesnelerle bağımlılıkları taklit etmeyi sağlar.



NEDEN ÖNEMLİ?

Service unit testlerinde repository veya dış servis mocklanır.

UYGULAMA / DİKKAT

Aşırı mocking tasarım kokusu olabilir; davranış değil sonuç test edilmelidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

73. Integration Testing

Security, Test ve Entegrasyon

KAVRAM

Birden fazla katmanın birlikte çalışmasını doğrular.



NEDEN ÖNEMLİ?

Testcontainers ile gerçek DB/Kafka gibi bağımlılıklar ayağa kaldırılabilir.

UYGULAMA / DİKKAT

Yavaş olduğu için seçici ve kritik senaryolara odaklı yazılmalıdır.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

74. Spring Cache

Security, Test ve Entegrasyon

KAVRAM

Sık kullanılan pahalı sonuçları tekrar hesaplamadan saklar.

NEDEN ÖNEMLİ?

@Cacheable, @CachePut, @CacheEvict temel annotation’lardır.

UYGULAMA / DİKKAT

Cache invalidation ve tutarlılık stratejisi net tasarlanmalıdır.



1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

75. Scheduling

Security, Test ve Entegrasyon

KAVRAM

Belirli zamanlarda çalışan işleri tanımlar.



NEDEN ÖNEMLİ?

@Scheduled cron, fixedDelay veya fixedRate ile kullanılır.

UYGULAMA / DİKKAT

Cluster ortamında aynı job'ın birden fazla node'da çalışması kontrol edilmelidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

76. Async Programming

Security, Test ve Entegrasyon

KAVRAM

Metotların ayrı thread üzerinde çalışmasını sağlar.



NEDEN ÖNEMLİ?

@Async ve TaskExecutor ile uygulanır.

UYGULAMA / DİKKAT

Transaction context, exception handling ve thread pool sınırları bilinmelidir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

77. Spring Mail

Security, Test ve Entegrasyon

KAVRAM

SMTP üzerinden mail gönderimini kolaylaştırır.

NEDEN ÖNEMLİ?

JavaMailSender, MimeMessage ve template entegrasyonu kullanılır.



UYGULAMA / DİKKAT

Production’da retry, queue ve rate limit düşünülmelidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

78. Spring WebSocket

Security, Test ve Entegrasyon

KAVRAM

Çift yönlü gerçek zamanlı iletişim sağlar.

NEDEN ÖNEMLİ?

Chat, canlı bildirim, dashboard update gibi senaryolarda kullanılır.

UYGULAMA / DİKKAT

STOMP, broker ve güvenlik yapılandırması önemlidir.



1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

79. Spring WebFlux ve Reactive Programming

Security, Test ve Entegrasyon

KAVRAM

Non-blocking reactive web stack’tir.

NEDEN ÖNEMLİ?

Mono ve Flux tipleriyle asenkron veri akışı temsil edilir.



UYGULAMA / DİKKAT

Yüksek eşzamanlılıkta faydalıdır; blocking kodla karıştırılırsa avantaj kaybolur.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

Modül 5

Boot, Cloud, AI ve Final Proje

80–100 numaralı konu başlıkları



Spring Boot, mikroservisler, observability, Spring AI ve uçtan uca üretim senaryosu.

80. Spring Boot'a Geçiş • 81. Auto Configuration • 82. Starter Dependencies • 83. Actuator • 84. Spring Boot + REST + JPA Uygulaması • 85. Katmanlı Mimari • 86. Controller – Service – Repository – Entity – DTO • 87. DTO / Mapper Yapısı • 88. SOLID ve Clean Code • 89. Design Patterns ile Spring • 90. Logging • 91. Docker ile Spring Uygulaması • 92. Production Deployment • 93. Spring Cloud'a Giriş • 94. Microservices Mimarisi • 95. API Gateway • 96. Service Discovery • 97. Config Server • 98. OpenFeign • 99. Circuit Breaker • 100. Messaging – Kafka / RabbitMQ

80. Spring Boot'a Geçiş

Boot, Cloud, AI ve Final Proje

KAVRAM

Framework bilgisini koruyarak Boot’un auto configuration ve starter kolaylıklarına geçiştir.

NEDEN ÖNEMLİ?

Boilerplate azalır, embedded server ile hızlı deploy edilir.



UYGULAMA / DİKKAT

Boot sihir değil, opinionated Spring yapılandırmasıdır.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

81. Auto Configuration

Boot, Cloud, AI ve Final Proje

KAVRAM

Classpath, bean varlığı ve properties'e göre otomatik bean yapılandırır.



NEDEN ÖNEMLİ?

Conditional annotation'lar temel mekanizmadır.

UYGULAMA / DİKKAT

Hangi auto config'in devrede olduğunu actuator/conditions ile analiz edebilirsiniz.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

82. Starter Dependencies

Boot, Cloud, AI ve Final Proje

KAVRAM

İlgili kütüphane setlerini tek dependency altında toplar.



NEDEN ÖNEMLİ?

spring-boot-starter-web, data-jpa, security gibi starterlar yaygındır.

UYGULAMA / DİKKAT

Versiyon uyumu Boot dependency management ile sağlanır.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

83. Actuator

Boot, Cloud, AI ve Final Proje

KAVRAM

Production-ready health, metrics, info ve diagnostics endpointleri sağlar.



NEDEN ÖNEMLİ?

/actuator/health, /metrics, /env gibi endpointler kullanılabilir.

UYGULAMA / DİKKAT

Güvenlik ve endpoint exposure ayarları production'da dikkat ister.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

84. Spring Boot + REST + JPA Uygulaması

Boot, Cloud, AI ve Final Proje

KAVRAM

Boot starter web + data-jpa + validation ile hızlı REST projesi kurulur.



NEDEN ÖNEMLİ?

Entity, repository, service, controller ve DTO zinciri uygulanır.

UYGULAMA / DİKKAT

Migration, logging, exception handling ve testler eklenince üretim kalitesi artar.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

85. Katmanlı Mimari

Boot, Cloud, AI ve Final Proje

KAVRAM

Uygulamayı sorumluluklara göre controller, service, repository gibi katmanlara ayırır.



NEDEN ÖNEMLİ?

Bağımlılıklar üstten alta akar; ters yönde sızıntı olmamalıdır.

UYGULAMA / DİKKAT

Test, bakım ve ekip çalışması için temel mimari disiplindir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

86. Controller – Service – Repository – Entity – DTO

Boot, Cloud, AI ve Final Proje

KAVRAM

Controller HTTP, service iş kuralı, repository veri erişimi, entity persistence modeli, DTO API modelidir.

NEDEN ÖNEMLİ?

Entity dış dünyaya doğrudan açılmamalıdır.



UYGULAMA / DİKKAT

Bu ayrım güvenlik, versiyonlama ve validasyon esnekliği sağlar.

1 DTO

2 Service

3 Tx

4 Repository

5 DB

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

87. DTO / Mapper Yapısı

Boot, Cloud, AI ve Final Proje

KAVRAM

DTO dış API sözleşmesini, mapper entity-DTO dönüşümünü yönetir.



NEDEN ÖNEMLİ?

MapStruct veya manuel mapper kullanılabilir.

UYGULAMA / DİKKAT

DTO kullanmamak entity sızıntısı ve lazy serialization hatası doğurabilir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

88. SOLID ve Clean Code

Boot, Cloud, AI ve Final Proje

KAVRAM

Single responsibility, open-closed, dependency inversion gibi prensipler Spring tasarımını güçlendirir.

NEDEN ÖNEMLİ?

Küçük service metotları, net interface ve anlamlı isimler bakım maliyetini düşürür.



UYGULAMA / DİKKAT

Framework bilgisi clean code yerine geçmez; onu destekler.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

89. Design Patterns ile Spring

Boot, Cloud, AI ve Final Proje

KAVRAM

Factory, Proxy, Singleton, Template Method, Strategy gibi desenler Spring içinde sıkça görülür.

NEDEN ÖNEMLİ?

AOP proxy, JdbcTemplate template method, BeanFactory factory örnekleridir.



UYGULAMA / DİKKAT

Desenleri tanımak framework davranışını daha hızlı kavratır.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

90. Logging

Boot, Cloud, AI ve Final Proje

KAVRAM

Uygulama davranışını izlenebilir hale getirir.



NEDEN ÖNEMLİ?

SLF4J facade, Logback implementasyonu yaygındır.

UYGULAMA / DİKKAT

Correlation id, log level ve structured logging production için kritiktir.

1 Tanım | 2 Container | 3 Bean/Proxy | 4 Test | 5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

91. Docker ile Spring Uygulaması

Boot, Cloud, AI ve Final Proje

KAVRAM

Spring uygulamasını container image olarak paketler.



NEDEN ÖNEMLİ?

Dockerfile, environment variables ve port mapping temel konulardır.

UYGULAMA / DİKKAT

JVM memory ayarı, layer caching ve image boyutu production’da önemlidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

92. Production Deployment

Boot, Cloud, AI ve Final Proje

KAVRAM

Uygulamayı güvenli, izlenebilir ve ölçeklenebilir şekilde canlıya alma sürecidir.



NEDEN ÖNEMLİ?

Profile, secret, migration, health check, rollback ve monitoring planı gerekir.

UYGULAMA / DİKKAT

CI/CD pipeline kalite kapılarını otomatikleştirmelidir.

1 Tanım

2 Container

3 Bean/Proxy

4 Test

5 Prod

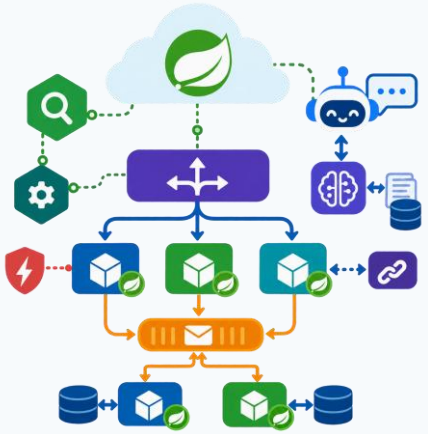
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

93. Spring Cloud'a Giriş

Boot, Cloud, AI ve Final Proje

KAVRAM

Dağıtık sistem problemleri için Spring tabanlı araçlar sunar.



NEDEN ÖNEMLİ?

Config, discovery, gateway, load balancing, circuit breaker gibi konuları kapsar.

UYGULAMA / DİKKAT

Mikroservis mimarisinde operasyonel karmaşıklığı yönetmeye yardım eder.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

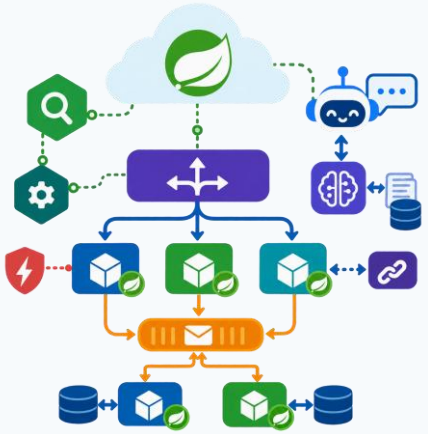
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

94. Microservices Mimarisi

Boot, Cloud, AI ve Final Proje

KAVRAM

Uygulamayı bağımsız deploy edilebilir küçük servisler halinde tasarlar.



NEDEN ÖNEMLİ?

Her servis kendi iş alanı ve verisine sahip olmalıdır.

UYGULAMA / DİKKAT

Dağıtık transaction, gözlemlenebilirlik ve network hataları temel zorluklardır.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

95. API Gateway

Boot, Cloud, AI ve Final Proje

KAVRAM

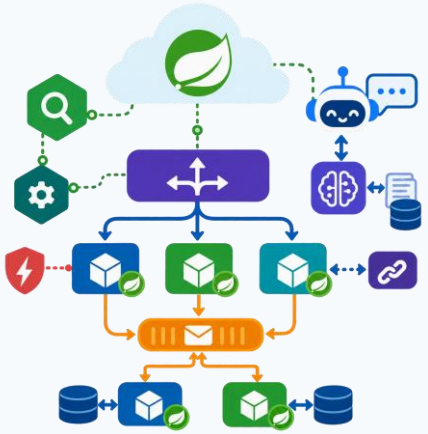
Client ile servisler arasında merkezi giriş noktasıdır.

NEDEN ÖNEMLİ?

Routing, auth, rate limit, logging ve cross-cutting policy uygular.

UYGULAMA / DİKKAT

Gateway iş kuralı taşımaz; trafik ve policy katmanıdır.



1 Client

2 Gateway

3 Service

4 Network

5 Observe

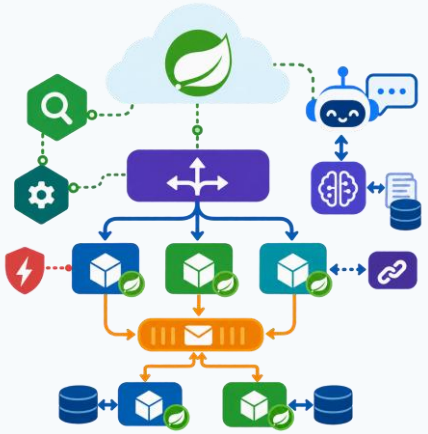
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

96. Service Discovery

Boot, Cloud, AI ve Final Proje

KAVRAM

Servislerin adreslerini dinamik olarak bulmasını sağlar.



NEDEN ÖNEMLİ?

Eureka, Consul veya Kubernetes service discovery kullanılabilir.

UYGULAMA / DİKKAT

Dinamik ölçekleme ve instance değişiminde manuel URL yönetimini kaldırır.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

97. Config Server

Boot, Cloud, AI ve Final Proje

KAVRAM

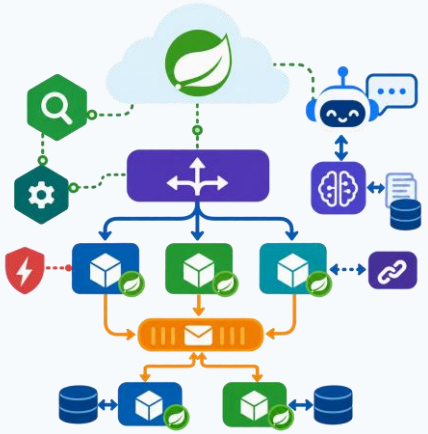
Dağıtık sistemlerde merkezi konfigürasyon yönetimi sağlar.

NEDEN ÖNEMLİ?

Git tabanlı config repository yaygın modeldir.

UYGULAMA / DİKKAT

Secret yönetimi ve refresh stratejisi güvenli tasarlanmalıdır.



1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

98. OpenFeign

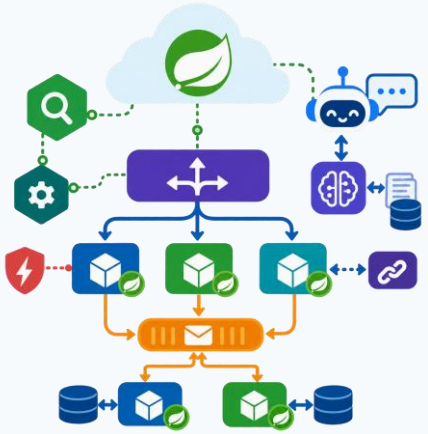
Boot, Cloud, AI ve Final Proje

KAVRAM

Declarative HTTP client yaklaşımıdır.

NEDEN ÖNEMLİ?

Interface yazarak servisler arası REST çağrısı yapılır.



UYGULAMA / DİKKAT

Timeout, retry, error decoder ve circuit breaker ile birlikte düşünülmelidir.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

99. Circuit Breaker

Boot, Cloud, AI ve Final Proje

KAVRAM

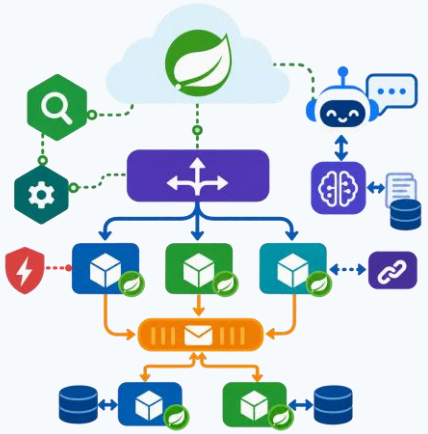
Hatalı veya yavaş servise çağrıları kontrollü keser.

NEDEN ÖNEMLİ?

Fail fast, fallback ve yarı açık durumlarla sistemi korur.

UYGULAMA / DİKKAT

Resilience4j Spring ekosisteminde sık kullanılan çözümdür.



1 Client

2 Gateway

3 Service

4 Network

5 Observe

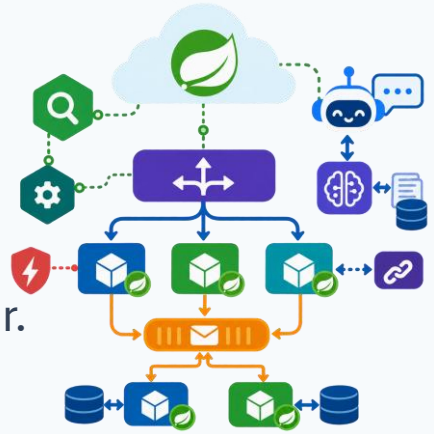
Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.

100. Messaging – Kafka / RabbitMQ

Boot, Cloud, AI ve Final Proje

KAVRAM

Servisler arası asenkron iletişim sağlar.



NEDEN ÖNEMLİ?

Kafka yüksek hacimli event streaming; RabbitMQ queue/routing senaryolarında güçlüdür.

UYGULAMA / DİKKAT

Idempotency, retry, dead-letter queue ve message schema kritik konulardır.

1 Client

2 Gateway

3 Service

4 Network

5 Observe

Kaynak notu: Resmi Spring Framework, Spring Boot ve Spring AI referansları temel alınarak hazırlanmıştır.